

Welcome Offer Access to everything. Now 30% off. [Upgrade now](#)

React Form Handling: 4 Approaches Every Developer Should Know



Devendra Dhote

Follow

3 min read · Jul 7, 2026





REACT FORM HANDLING: 4 Approaches Every Developer Should Know

One of the first challenges I faced while learning React was handling forms.

At first, everything looked simple. But as soon as the number of input fields started increasing, I realized that not every approach scales well.

While practicing, I came across three different ways of managing form data. Each approach solves the same problem, but each one improves upon the previous one.

Here's what I learned.

. . .

1. The Brute Force Approach

This is the approach almost every beginner starts with.

Create a separate state for every input.

```
const [name, setName] = useState("");  
const [email, setEmail] = useState("");  
const [password, setPassword] = useState("");
```

Then every input gets its own `onChange` handler.

```
<input  
  value={name}  
  onChange={(e) => setName(e.target.value)}  
>
```

```
<input
  value={email}
  onChange={(e) => setEmail(e.target.value)}
/>
```

Why it works

- Easy to understand
- Great for learning `useState`
- Good for very small forms

The problem

Imagine building a registration form with 15–20 fields.

Now you'll have:

- 20 state variables
- 20 setter functions
- 20 `onChange` handlers

The code quickly becomes repetitive and difficult to maintain.

. . .

2. A Better Approach

Instead of creating multiple states, we can keep the entire form inside a single object.

```
const [formData, setFormData] = useState({  
  name: "",  
  email: "",  
  password: "",  
});
```

Now each input updates only its corresponding property.

```
<input  
  value={formData.name}  
  onChange={(e) =>  
    setFormData({  
      ...formData,  
      name: e.target.value,  
    })  
  }  
>
```

```
}  
</>
```

For the email field:

```
<input  
  value={formData.email}  
  onChange={(e) =>  
    setFormData({  
      ...formData,  
      email: e.target.value,  
    })  
  }  
>
```

Why it's better

- Only one state to manage
- Cleaner data structure
- Easier to submit the entire form

But...

We're still repeating almost the same `onChange` logic for every input.



That repetition bothered me.

. . .

3. The DRY (Don't Repeat Yourself) Approach

Although the previous approach is much cleaner, we can still improve it by removing repetitive `onChange` handlers.

Instead of writing separate logic for every input, we give each input a unique `name` attribute and use a single `handleChange()` function.

```
<input  
  name="name"  
  value={formData.name}
```

```
    onChange={handleChange}  
  />
```

```
<input  
  name="email"  
  value={formData.email}  
  onChange={handleChange}  
/>
```

```
const handleChange = (e) => {  
  const { name, value } = e.target;
```

```
  setFormData((prev) => ({  
    ...prev,  
    [name]: value,  
  }));  
};
```

The interesting part is this line:

```
[name]: value
```


This is called a **Computed Property Name**. It allows JavaScript to dynamically update the correct property based on the input's `name` attribute.

With this approach, one function can handle every input field, making the code much cleaner and following the **DRY (Don't Repeat Yourself)** principle.

. . .

4. The Optimized Approach (Using `useRef`)

While exploring React further, I discovered another way of handling forms using the `useRef` hook.

Instead of storing every keystroke inside React state, we can directly access the input elements using refs.

```
const formRef = useRef({});
```

Each input registers itself inside the ref object.

```
<input
  type="text"
  placeholder="Name"
  ref={(element) => (formRef.current.name = element)}
/>
```

```
<input
  type="email"
  placeholder="Email"
  ref={(element) => (formRef.current.email = element)}
/>
```

Now, when the form is submitted, we can directly read the values from the DOM.

```
const handleSubmit = (e) => {
  e.preventDefault();
```

```
  const data = {
    name: formRef.current.name.value,
    email: formRef.current.email.value,
  };
  console.log(data);
};
```

This approach avoids updating React state on every keystroke because the browser manages the input values until they're needed.

Interestingly, this is also the core idea behind libraries like **React Hook Form**, which rely heavily on refs and uncontrolled components to achieve better performance and fewer re-renders.

. . .

My Learning Journey

Looking back, my understanding of form handling evolved like this:

```
Brute Force  
(Multiple States)  
  ↓  
Better Approach  
(Single State Object)  
  ↓  
DRY Approach  
(Single State + One Dynamic handleChange)  
  ↓  
Optimized Approach  
(useRef + Uncontrolled Inputs)
```

Each approach solves a real problem introduced by the previous one. Understanding this progression helped me appreciate not just *how* to build forms in React, but *why* different solutions exist.

. . .

Final Thoughts

One thing I enjoy about React is that it doesn't force a single way of solving a problem.

You can start with the simplest solution, understand its limitations, and gradually move toward a cleaner and more scalable approach.

That's exactly how I learned form handling — and honestly, understanding *why* each approach exists taught me much more than simply memorizing the final solution.

Happy coding champs ❤️

React ✓

JavaScript ✓

Web Development ✓

Programming ✓